receptionist and send messages. By default, the strategy in a group router is `withRandomRouting`.

Creating a group router is easier than creating a pool router. You just need to pass the `ServiceKey` of the routees. You don't have to specify the number of routees because that is defined by the routees registered with that key.

Let's say you have a photo processor like the following:

```
object PhotoProcessor {
  val key = ServiceKey[String]("photo-processor-key")
  def apply(): Behavior[String] = ...
}
```

You can create a camera that can send the content of its photos to a pool of `PhotoProcessor`s like this:

```
object Camera {

  final case class Photo(url: String)

  def apply() = Behaviors.setup[Photo] { context =>
    val routingBehavior: GroupRouter[String] =
      Routers.group(PhotoProcessor.key).withRoundRobinRouting()     ❶
    val router: ActorRef[String] =
      context.spawn(routingBehavior, "photo-processor-pool")        ❷

    Behaviors.receiveMessage {
      case Photo(url) =>
          pool ! url                                                ❸
          Behaviors.same
    }
}}
```

❶ Sets the PhotoProcessor as the routee type

❷ Spawns the router

❸ Sends photos to available routees in a round-robin fashion